
Java is a **programming language** and **platform** independent language. Java is a high level, robust, object-oriented and secure programming language.

Java was developed by Sun Microsystems (which is now the subsidiary of Oracle) in the year 1995. James Gosling is known as the father of Java. Before Java, its name was Oak. Since Oak was already a registered company, so James Gosling and his team changed the name from Oak to Java.

Platform: Any hardware or software environment in which a program runs, is known as a platform. Since Java has a runtime environment (JRE) and API, it is called a platform.

Types of Java Applications

There are mainly 4 types of applications that can be created using Java programming:

1) Standalone Application

Standalone applications are also known as desktop applications or window-based applications. These are traditional software that we need to install on every machine. Examples of standalone application are Media player, antivirus, etc. AWT and Swing are used in Java for creating standalone applications.

2) Web Application

An application that runs on the server side and creates a dynamic page is called a web application. Currently, Servlet, JSP, Struts, Spring, Hibernate, JSF, etc. technologies are used for creating web applications in Java.

3) Enterprise Application

An application that is distributed in nature, such as banking applications, etc. is called an enterprise application. It has advantages like high-level security, load balancing, and clustering. In Java, EJB is used for creating enterprise applications.

4) Mobile Application

An application which is created for mobile devices is called a mobile application. Currently, Android and Java ME are used for creating mobile applications.

History of Java

1) **James Gosling, Mike Sheridan, and Patrick Naughton** initiated the Java language project in June 1991. The small team of sun engineers called **Green Team**.

2) Initially it was designed for small, embedded systems in electronic appliances like set-top boxes.

3) Firstly, it was called "**Greentalk**" by James Gosling, and the file extension was .gt.

4) After that, it was called **Oak** and was developed as a part of the Green project.

Oak is a symbol of strength and chosen as a national tree of many countries like the U.S.A., France, Germany, Romania, etc.

6) In 1995, Oak was renamed as "**Java**" because it was already a trademark by Oak Technologies.

According to James Gosling, "Java was one of the top choices along with **Silk**". Since Java was so unique, most of the team members preferred Java than other names.

8) Java is an island in Indonesia where the first coffee was produced (called Java coffee). It is a kind of espresso bean. Java name was chosen by James Gosling while having a cup of coffee nearby his office.

9) Initially developed by James Gosling at Sun Microsystems (which is now a subsidiary of Oracle Corporation) and released in 1995.

10) In 1995, Time magazine called **Java one of the Ten Best Products of 1995**.

11) JDK 1.0 was released on January 23, 1996. After the first release of Java, there have been many additional features added to the language. Now Java is being used in Windows applications, Web applications, enterprise applications, mobile applications, cards, etc. Each new version adds new features in Java and current new version is JDK 21.

Features of Java

Simple

Java is very easy to learn, and its syntax is simple, clean and easy to understand. According to Sun Microsystem, Java language is a simple programming language because:

- Java syntax is based on C++ (so easier for programmers to learn it after C++).

- Java has removed many complicated and rarely-used features, for example, explicit pointers, operator overloading, etc.
- There is no need to remove unreferenced objects because there is an Automatic Garbage Collection in Java.

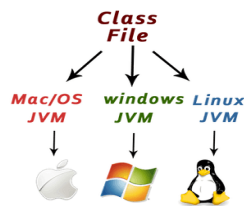
Object-oriented

Java is an object-oriented programming language. Everything in Java is an object. Object-oriented means we organize our software as a combination of different types of objects that incorporate both data and behavior.

Object-oriented programming (OOPs) is a methodology that simplifies software development and maintenance by providing some rules.

Basic concepts of OOPs are: Object, Class, Inheritance, Polymorphism, Abstraction, Encapsulation

Platform Independent



A platform is the hardware or software environment in which a program runs. There are two types of platforms software-based and hardware-based. Java provides a software-based platform.

Java code can be executed on multiple platforms, for example, Windows, Linux, Sun Solaris, Mac/OS, etc. Java code is compiled by the compiler and converted into bytecode. This bytecode is a platform-independent code because it can be run on multiple platforms, i.e., Write Once and Run Anywhere (WORA).

Secured

Java is best known for its security. With Java, we can develop virus-free systems. Java is secured because:

- **No explicit pointer**
- **Java Programs run inside a virtual machine sandbox**

Robust

The English meaning of Robust is strong. Java is robust because:

- It uses strong memory management.
- There is a lack of pointers that avoids security problems.
- Java provides automatic garbage collection which runs on the Java Virtual Machine to get rid of objects which are not being used by a Java application anymore.

Architecture-neutral

Java is architecture neutral because there are no implementation dependent features, for example, the size of primitive types is fixed.

In C programming, int data type occupies 2 bytes of memory for 32-bit architecture and 4 bytes of memory for 64-bit architecture. However, it occupies 4 bytes of memory for both 32 and 64-bit architectures in Java.

Portable

Java is portable because it facilitates you to carry the Java byte code to any platform. It doesn't require any implementation.

High-performance

Java is faster than other traditional interpreted programming languages because Java bytecode is "close" to native code. It is still a little bit slower than a compiled language (e.g., C++). Java is an interpreted language that is why it is slower than compiled languages, e.g., C, C++, etc.

Distributed

Java is distributed because it facilitates users to create distributed applications in Java. RMI and EJB are used for creating distributed applications. This feature of Java makes us able to access files by calling the methods from any machine on the internet.

Multi-threaded

A thread is like a separate program, executing concurrently. We can write Java programs that deal with many tasks at once by defining multiple threads. The main advantage of multi-threading is that it doesn't occupy memory for each thread. It shares a common memory area. Threads are important for multi-media, Web applications, etc.

Dynamic

Java is a dynamic language. It supports the dynamic loading of classes. It means classes are loaded on demand. It also supports functions from its native languages, i.e., C and C++.

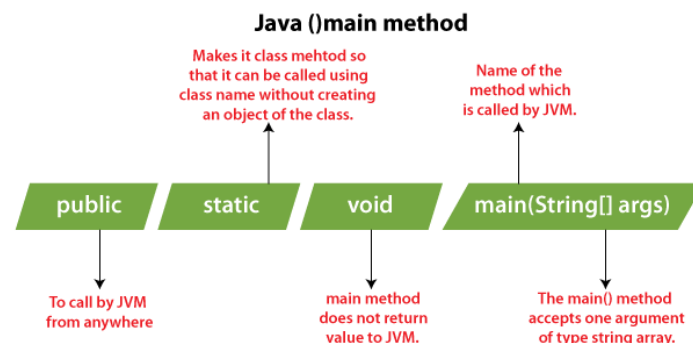
Java supports dynamic compilation and automatic memory management (garbage collection).

Java Example

Let's have a quick look at Java programming example. A detailed description of Hello Java example is available in next page.

class Simple

```
{  
    public static void main(String args[])  
{  
    System.out.println("Hello Java");  
}  
}
```



Let's divide the syntax of the **main()** method into several parts and understand each one of them:

1) public

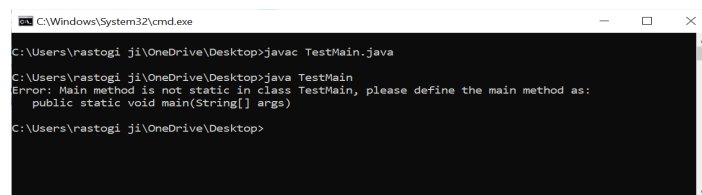
It is not so complicated to understand. It is an access modifier of the **main()** method. We create `main()` method with **public** access specifier to execute it by any program. So, it is required to define `main()` method public and if we define the `main()` method as non-public, it will throw the following error:

MODULE-1

// create TestMain class to check error for non-public main() method

```
public class TestMain {  
    // define main() method as non-public  
    static void main(String[] args){  
        System.out.println("Non-public main method.");  
    }  
}
```

Output:



```
C:\Windows\System32\cmd.exe  
C:\Users\rastogi_ji\OneDrive\Desktop>javac TestMain.java  
C:\Users\rastogi_ji\OneDrive\Desktop>java TestMain  
Error: Main method is not static in class TestMain, please define the main method as:  
    public static void main(String[] args)  
C:\Users\rastogi_ji\OneDrive\Desktop>
```

2) static

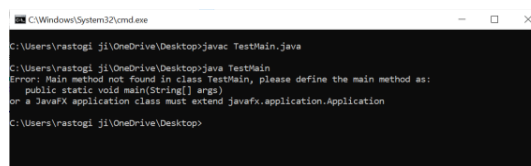
The **static** is a keyword which we use in the main() method to define it as static. There is no object of the class available at the time of starting java runtime, and because of that, we have to define the main() method as **static**. By doing that, JVM can load the class into the main memory and call the main() method.

So, if we define main() method as non-static method, JVM would not be able to call it and throws the following error:

// create TestMain class to check error for non-static main() method

```
public class TestMain {  
    // define main() method as non-public  
    public void main(String[] args){  
        System.out.println("Non-public main method.");  
    }  
}
```

Output:



```
C:\Windows\System32\cmd.exe  
C:\Users\rastogi_ji\OneDrive\Desktop>javac TestMain.java  
C:\Users\rastogi_ji\OneDrive\Desktop>java TestMain  
Error: Main method not found in class TestMain, please define the main method as:  
    public static void main(String[] args)  
    or a JavaFX application class must extend javafx.application.Application  
C:\Users\rastogi_ji\OneDrive\Desktop>
```

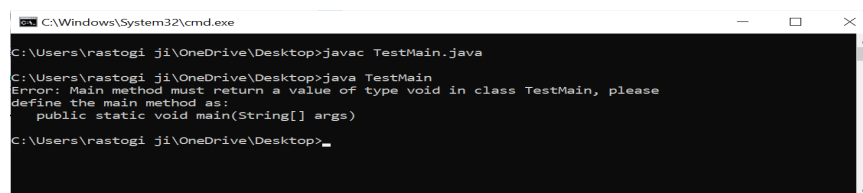
3) void

As we know that, each method provides some return type such as String, Boolean, Integer etc. The Java main() method doesn't return anything, and its return type is **void**. The main() method doesn't return anything to make things simple. The program will be terminated after executing the **main()** method, and returning anything from the main() method is worthless because JVM will be doing nothing for the returned object.

If we return something from the main() method, it will throw the following error:

1. // create TestMain class to check error when it doesn't have a return type
2. **public class** TestMain {
3. // define main() method without return type
4. **public static** main(String[] args){
5. System.out.println("Main method.");
6. }
7. }

Output:



```
C:\Windows\System32\cmd.exe
C:\Users\rastogi ji\OneDrive\Desktop>javac TestMain.java
C:\Users\rastogi ji\OneDrive\Desktop>java TestMain
Error: Main method must return a value of type void in class TestMain, please
define the main method as:
  public static void main(String[] args)
C:\Users\rastogi ji\OneDrive\Desktop>
```

4) main()

It is the name of the **main()** method. The name of the method is static and we cannot change it. If we try to change the name of the method(), it will throw the following error:

// create TestMain class to check error for main() method having different name

```
public class TestMain {
    // define main() method as non-public
    public static int newMain(String[] args){
        System.out.println("Main method.");
    }
}
```

Output:

```
C:\Windows\System32\cmd.exe
C:\Users\rastogi_ji\OneDrive\Desktop>javac TestMain.java
C:\Users\rastogi_ji\OneDrive\Desktop>java TestMain
Error: Main method not found in class TestMain, please define the main method as:
    public static void main(String[] args)
or a JavaFX application class must extend javafx.application.Application
C:\Users\rastogi_ji\OneDrive\Desktop>_
```

5) String[] args or String args[]

The Java main() method takes a single argument of type String array as a parameter also referred to as a **command-line argument**.

Reasons for defining main() method as static

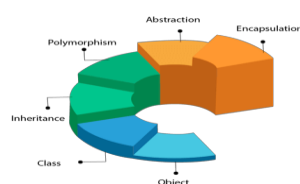
We cannot call a method without creating an instance of its class, and we already told you before that at the time of starting JVM, there is no object of a class. We create the main() method as static so that JVM can load the class into the main memory.

- The main() method is the entry point of each and every Java program. The main() method is required because the compiler starts executing a program from this entry point.
- The JVM needs to instantiate the class if the main() method is allowed to be non-static.
- JVM can call the static methods easily without creating an instance of the class by using the class name only.
- As discussed above, the main() method should be public, static, and have a return type void. If we do not define it as public and static or return something from the method, it will definitely throw an error.

Java OOPs Concepts

The main aim of object-oriented programming is to implement real-world entities, for example, object, classes, abstraction, inheritance, polymorphism, etc.

OOPs (Object-Oriented Programming System)



Object

Any entity that has state and behavior is known as an object. For example, a chair, pen, table, keyboard, bike, etc.



Example: A dog is an object because it has states like color, name, breed, etc. as well as behaviors like wagging the tail, barking, eating, etc.

Class

Collection of objects is called class. A class can also be defined as a blueprint from which you can create an individual object. Class doesn't consume any space.

Inheritance

When one object acquires all the properties and behaviors of a parent object, it is known as inheritance. It provides code reusability. It is used to achieve runtime polymorphism.

Polymorphism

If one task is performed in different ways, it is known as polymorphism. For example: to convince the customer differently, to draw something, for example, shape, triangle, rectangle, etc.

In Java, we use method overloading and method overriding to achieve polymorphism.

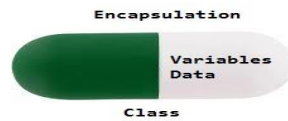
Another example can be to speak something; for example, a cat speaks meow, dog barks woof, etc.

Abstraction

Hiding internal details and showing functionality is known as abstraction. For example phone call, we don't know the internal processing.

MODULE-1

Encapsulation

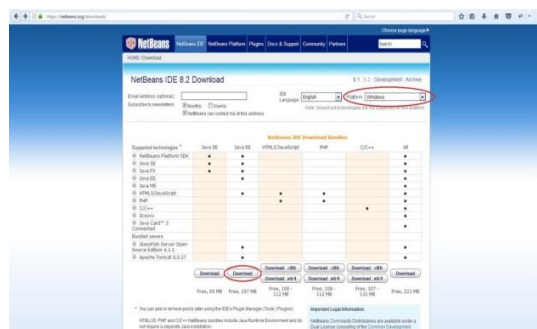


Binding (or wrapping) code and data together into a single unit are known as encapsulation. For example, a capsule, it is wrapped with different medicines.

A java class is the example of encapsulation. Java bean is the fully encapsulated class because all the data members are private here.

Creating, Compiling, and Executing a Simple Java Program

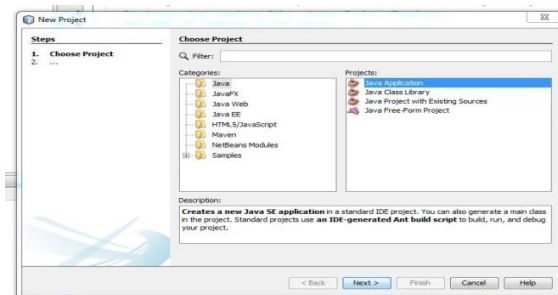
Install the Netbeans IDE and start it.



Steps to develop a Java program in Netbeans IDE

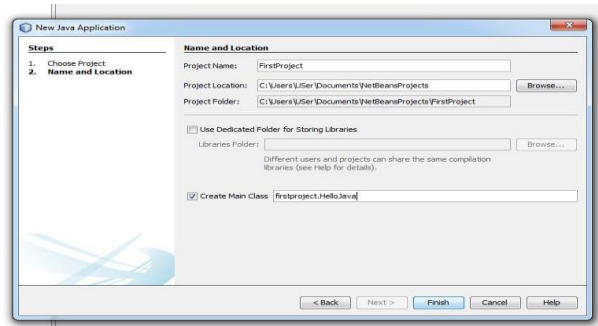
Below are the simple steps to develop a java program using Netbeans IDE.

Step 1: Start the Netbeans and click on **File --> New --> New Project** (or press **Ctrl + Shift + N**) select java under categories and Java Application under projects--> **Next**



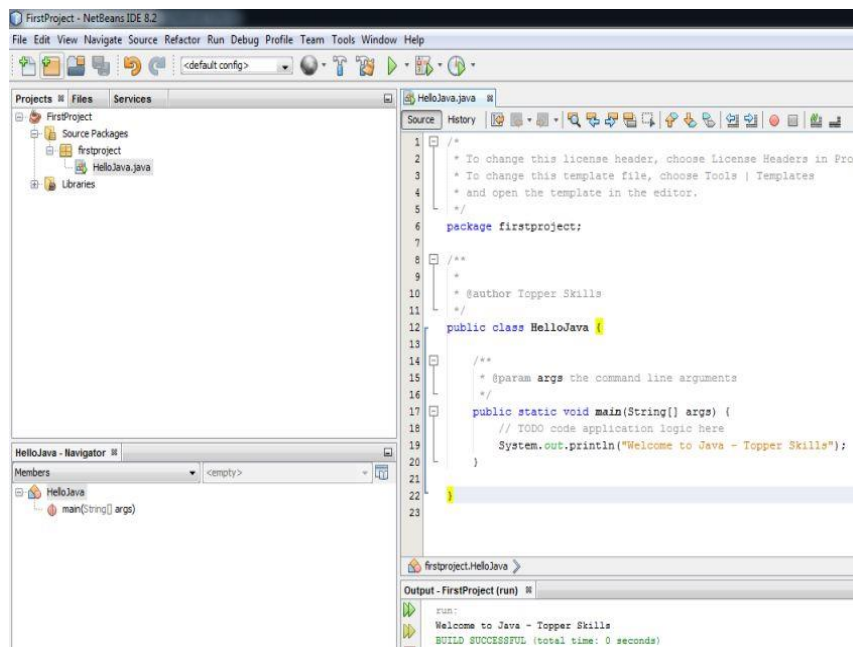
MODULE-1

Step 2: Enter the project name and main class name --> Finish



Step 3: Write the Java program source code.

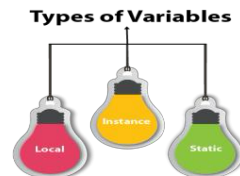
Step 4: Execute the program Click on Run --> Run Project (or press F6) You will see the output.



Java Variables

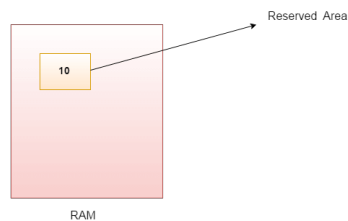
A variable is a container which holds the value while the Java program is executed. A variable is assigned with a data type.

Variable is a name of memory location. There are three types of variables in java: local, instance and static.



Variable

A variable is the name of a reserved area allocated in memory. In other words, it is a name of the memory location. It is a combination of "vary + able" which means its value can be changed.



`int data=50;`//Here data is variable

1) Local Variable

A variable declared inside the body of the method is called local variable. You can use this variable only within that method and the other methods in the class aren't even aware that the variable exists.

A local variable cannot be defined with "static" keyword.

2) Instance Variable

A variable declared inside the class but outside the body of the method, is called an instance variable. It is not declared as static.

It is called an instance variable because its value is instance-specific and is not shared among instances.

3) Static variable

A variable that is declared as static is called a static variable. It cannot be local. You can create a single copy of the static variable and share it among all the instances of the class. Memory allocation for static variables happens only once when the class is loaded in the memory.

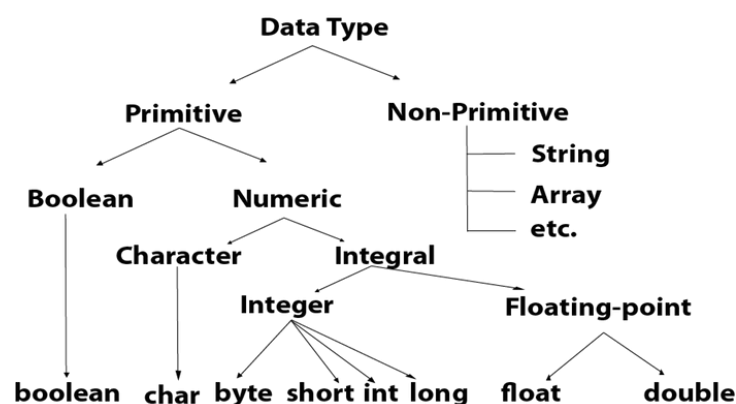
Example to understand the types of variables in java

```
public class A
{
    static int m=100;//static variable
    void method()
    {
        int n=90;//local variable
    }
    public static void main(String a rgs[])
    {
        int data=50;//instance variable
    }
} //end of class
```

Data Types in Java

Data types specify the different sizes and values that can be stored in the variable. There are two types of data types in Java:

1. **Primitive data types:** The primitive data types include boolean, char, byte, short, int, long, float and double.
2. **Non-primitive data types:** The non-primitive data types include Classes, Interfaces, and Arrays.



MODULE-1

Data Type	Default size
boolean	1 bit
char	2 byte
byte	1 byte
short	2 byte
int	4 byte
long	8 byte
float	4 byte
double	8 byte

An **interface in Java** is a blueprint of a class. It has static constants and abstract methods.

The interface in Java is *a mechanism to achieve abstraction*. There can be only abstract methods in the Java interface, not method body. It is used to achieve abstraction and multiple inheritance in Java.

In other words, you can say that interfaces can have abstract methods and variables. It cannot have a method body.

Operators in Java

Operator in Java is a symbol that is used to perform operations. For example: +, -, *, / etc.

There are many types of operators in Java which are given below:

Operator Type	Category	Precedence
Unary	postfix	<i>expr++ expr--</i>
	prefix	<i>++expr --expr +expr -expr ~ !</i>
Arithmetic	multiplicative	<i>* / %</i>
	additive	<i>+ -</i>

MODULE-1

Relational	comparison	< > <= >= instanceof
	equality	== !=
Logical	logical AND	&&
	logical OR	
Ternary	ternary	? :
Assignment	assignment	= += -= *= /= %= &= ^= = <<= >>= >>>=

Java Unary Operator Example: ++ and --

```
public class OperatorExample{
    public static void main(String args[]){
        int x=10;
        System.out.println(x++); //10 (11)
        System.out.println(++x); //12
        System.out.println(x--); //12 (11)
        System.out.println(--x); //10
    }
}
```

Output:

```
10
12
12
10
```

Java Unary Operator Example 2: ++ and --

```
public class OperatorExample{
    public static void main(String args[]){
        int a=10;
        int b=10;
        System.out.println(a++ + ++a); //10+12=22
        System.out.println(b++ + b++); //10+11=21
    }
}
```

Output:

```
22
21
```

Java Arithmetic Operator Example

```
public class OperatorExample{
    public static void main(String args[]){
        int a=10;
        int b=5;
        System.out.println(a+b);//15
        System.out.println(a-b);//5
        System.out.println(a*b);//50
        System.out.println(a/b);//2
        System.out.println(a%b);//0
    }
}
```

Output:

```
15
5
50
2
0
```

Java Arithmetic Operator Example: Expression

```
public class OperatorExample{
    public static void main(String args[])
    {
        System.out.println(10*10/5+3-1*4/2);
    }
}
```

Output:

```
21
```

Java AND Operator Example: Logical && and Bitwise &

The logical && operator doesn't check the second condition if the first condition is false. It checks the second condition only if the first one is true.

The bitwise & operator always checks both conditions whether first condition is true or false.

```
public class OperatorExample{
    public static void main(String args[]){
        int a=10;
        int b=5;
        int c=20;
        System.out.println(a<b&&a<c);//false && true = false
        System.out.println(a<b&a<c);//false & true = false
    }
}
```

Output:

false

false

Java Ternary Operator

Java Ternary operator is used as one line replacement for if-then-else statement and used a lot in Java programming. It is the only conditional operator which takes three operands.

Java Ternary Operator Example

```
public class OperatorExample{  
    public static void main(String args[]){  
  
        int a=2;  
        int b=5;  
        int min=(a<b)?a:b;  
        System.out.println(min);  
    }  
}
```

Output:

2

Java Assignment Operator Example

```
public class OperatorExample{  
    public static void main(String args[]){  
        int a=10;  
        int b=20;  
        a+=4;//a=a+4 (a=10+4)  
        b-=4;//b=b-4 (b=20-4)  
        System.out.println(a);  
        System.out.println(b);  
    }  
}
```

Output:

14

16

Reading Input from the Console using Scanner class

Java Scanner Class

Java **Scanner** class allows the user to take input from the console. It belongs to **java.util** package. It is used to read the input of primitive types like int, double, long, short, float, and byte. It is the easiest way to read input in Java program.

Syntax

Scanner sc=**new** Scanner(System.in);

The above statement creates a constructor of the Scanner class having **System.in** as an argument. It means it is going to read from the standard input stream of the program. The **java.util** package should be import while using Scanner class.

Methods of Java Scanner Class

Java Scanner class provides the following methods to read different primitives types:

Method	Description
int nextInt()	It is used to scan the next token of the input as an integer.
float nextFloat()	It is used to scan the next token of the input as a float.
double nextDouble()	It is used to scan the next token of the input as a double.
byte nextByte()	It is used to scan the next token of the input as a byte.
String nextLine()	Advances this scanner past the current line.
boolean nextBoolean()	It is used to scan the next token of the input into a boolean value.
long nextLong()	It is used to scan the next token of the input as a long.
short nextShort()	It is used to scan the next token of the input as a Short.
BigInteger nextBigInteger()	It is used to scan the next token of the input as a BigInteger.

BigDecimal nextBigDecimal()	It is used to scan the next token of the input as a BigDecimal.
--	---

Example of integer input from user

The following example allows user to read an integer form the System.in.

```
import java.util.*;
class UserInputDemo
{
    public static void main(String[] args)
    {
        Scanner sc= new Scanner(System.in); //System.in is a standard input stream
        System.out.print("Enter first number- ");
        int a= sc.nextInt();
        System.out.print("Enter second number- ");
        int b= sc.nextInt();
        System.out.print("Enter third number- ");
        int c= sc.nextInt();
        int d=a+b+c;
        System.out.println("Total= " +d);
    } }
```

Output:



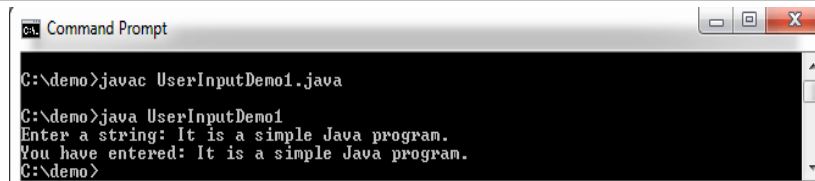
```
Command Prompt
C:\demo>javac UserInputDemo.java
C:\demo>java UserInputDemo
Enter first number- 6
Enter second number- 44
Enter third third- 23
Total= 73
C:\demo>
```

Example of String Input from user

Let's see another example, in which we have taken string input.

```
import java.util.*;
class UserInputDemo1
{
    public static void main(String[] args)
    {
        Scanner sc= new Scanner(System.in); //System.in is a standard input stream
        System.out.print("Enter a string: ");
        String str= sc.nextLine(); //reads string
        System.out.print("You have entered: "+str);
    } }
```

Output:



```
C:\demo>javac UserInputDemo1.java
C:\demo>java UserInputDemo1
Enter a string: It is a simple Java program.
You have entered: It is a simple Java program.
C:\demo>
```

Type Casting and Type Conversion

The process of converting the value of one data type (int, float, double, etc.) to another data type is known as typecasting.

Syntax:

Destination_datatype = (target_datatype) variable;

(data_type) it is known as casting operator

There are two major types.

1. Widening Type Casting

Converting int to double

```
class Main {
    public static void main(String[] args) {
        // create int type variable
        int num = 10;
        System.out.println("The integer value: " + num);

        // convert into double type
        double data = num;
        System.out.println("The double value: " + data);
    }
}
```

Output

```
The integer value: 10
The double value: 10.0
```

In the above example, we are assigning the int type variable named num to a double type variable named data.

Here, the Java first converts the int type data into the double type. And then assign it to the double variable.

In the case of Widening Type Casting, the lower data type (having smaller size) is converted into the higher data type (having larger size). Hence there is no loss in data. This is why this type of conversion happens automatically.

2. Narrowing Type Casting

In **Narrowing Type Casting**, we manually convert one data type into another using the parenthesis.

Converting double into an int

```
class Main {  
    public static void main(String[] args) {  
        // create double type variable  
        double num = 10.99;  
        System.out.println("The double value: " + num);  
  
        // convert into int type  
        int data = (int)num;  
        System.out.println("The integer value: " + data);  
    }  
}
```

Output

```
The double value: 10.99  
The integer value: 10
```

In the above example, we are assigning the `double` type variable named `num` to an `int` type variable named `data`.

Notice the line,

`int data = (int)num;`

Here, the `int` keyword inside the parenthesis indicates that that the `num` variable is converted into the `int` type.

In the case of Narrowing Type Casting, the higher data types (having larger size) are converted into lower data types (having smaller size). Hence there is the loss of data. This is why this type of conversion does not happen automatically.

Note: This is also known as Explicit Type Casting.

Example 1: Type conversion from int to String

```
class Main {  
    public static void main(String[] args) {  
        // create int type variable  
        int num = 10;  
        System.out.println("The integer value is: " + num);  
  
        // converts int to string type  
        String data = String.valueOf(num);  
        System.out.println("The string value is: " + data);  
    }  
}
```

Output

```
The integer value is: 10  
The string value is: 10
```

In the above program, notice the line

```
String data = String.valueOf(num);
```

Here, we have used the `valueOf()` method of the Java String class to convert the int type variable into a string.

Example 2: Type conversion from String to int

```
class Main {  
    public static void main(String[] args) {  
        // create string type variable  
        String data = "10";  
        System.out.println("The string value is: " + data);  
  
        // convert string variable to int  
        int num = Integer.parseInt(data);
```

```
System.out.println("The integer value is: " + num);  
}  
}
```

Output

```
The string value is: 10  
The integer value is: 10
```

In the above example, notice the line

```
int num = Integer.parseInt(data);
```

Here, we have used the `parseInt()` method of the Java `Integer` class to convert a string type variable into an `int` variable.

Control Statements in Java

Java provides three types of control flow statements.

1. Decision Making statements
 - if statements
 - switch statement
2. Loop statements
 - do while loop
 - while loop
 - for loop
 - for-each loop
3. Jump statements
 - break statement
 - continue statement

Decision-Making statements:

As the name suggests, decision-making statements decide which statement to execute and when. Decision-making statements evaluate the Boolean expression and control the program flow depending upon the result of the condition provided. There are two types of decision-making statements in Java, i.e., If statement and switch statement.

1) If Statement:

In Java, the "if" statement is used to evaluate a condition. The control of the program is diverted depending upon the specific condition. The condition of the If statement gives a Boolean value, either true or false. In Java, there are four types of if-statements given below.

1. Simple if statement
2. if-else statement
3. if-else-if ladder
4. Nested if-statement

Let's understand the if-statements one by one.

1) Simple if statement:

It is the most basic statement among all control flow statements in Java. It evaluates a Boolean expression and enables the program to enter a block of code if the expression evaluates to true.

Syntax of if statement is given below.

```
if(condition)
{
statement 1; //executes when condition is true
}
```

Consider the following example in which we have used the if statement in the java code.

```
public class Student {
public static void main(String[] args) {
int x = 10;
int y = 12;
if(x+y > 20) {
System.out.println("x + y is greater than 20");
}
}
}
```

Output:

```
x + y is greater than 20
```

2) if-else statement

The if-else statement is an extension to the if-statement, which uses another block of code, i.e., else block. The else block is executed if the condition of the if-block is evaluated as false.

Syntax:

```
if(condition) {  
    statement 1; //executes when condition is true  
}  
else{  
    statement 2; //executes when condition is false  
}
```

Consider the following example.

```
public class Student {  
    public static void main(String[] args) {  
        int x = 10;  
        int y = 12;  
        if(x+y < 10) {  
            System.out.println("x + y is less than 10");  
        } else {  
            System.out.println("x + y is greater than 20");  
        }  
    }  
}
```

Output:

```
x + y is greater than 20
```

3) if-else-if ladder:

The if-else-if statement contains the if-statement followed by multiple else-if statements. In other words, we can say that it is the chain of if-else statements that create a decision tree where the program may enter in the block of code where the condition is true. We can also define an else statement at the end of the chain.

Syntax of if-else-if statement is given below.

```
if(condition 1) {  
    statement 1; //executes when condition 1 is true  
}  
else if(condition 2) {
```

```
statement 2; //executes when condition 2 is true
}
else {
statement 2; //executes when all the conditions are false
}
```

Consider the following example.

```
Student.java
public class Student {
public static void main(String[] args) {
String city = "Delhi";
if(city == "Meerut") {
System.out.println("city is meerut");
}else if (city == "Noida") {
System.out.println("city is noida");
}else if(city == "Agra") {
System.out.println("city is agra");
}else {
System.out.println(city);
}
}
}
```

Output:

```
Delhi
```

4. Nested if-statement

In nested if-statements, the if statement can contain a **if** or **if-else** statement inside another if or else-if statement.

Syntax of Nested if-statement is given below.

```
if(condition 1) {
statement 1; //executes when condition 1 is true
if(condition 2) {
statement 2; //executes when condition 2 is true
}
else{
statement 2; //executes when condition 2 is false
}
}
```

Consider the following example.

```
import java.util.*;
import java.lang.*;
import java.io.*;

class Nested
{
    public static void main(String args[])
    {
        int a=10;
        int b=20;

        if(a==10){
            if(b==20){
                System.out.println("Nested if statement works");
            }
        }
    }
}
```

Output: Nested if statement works

Switch Statement:

In Java, Switch statements are similar to if-else-if statements. The switch statement contains multiple blocks of code called cases and a single case is executed based on the variable which is being switched. The switch statement is easier to use instead of if-else-if statements. It also enhances the readability of the program.

Points to be noted about switch statement:

- The case variables can be int, short, byte, char. String type is also supported since version 7 of Java
- Cases cannot be duplicate
- Default statement is executed when any of the case doesn't match the value of expression. It is optional.
- Break statement terminates the switch block when the condition is satisfied. It is optional, if not used, next case is executed.
- While using switch statements, we must notice that the case expression will be of the same type as the variable. However, it will also be a constant value.

The syntax to use the switch statement is given below.

```
switch (expression){  
    case value1:  
        statement1;  
        break;  
    .  
    .  
    .  
    case valueN:  
        statementN;  
        break;  
    default:  
        default statement;  
}
```

Consider the following example to understand the flow of the switch statement.

```
public class Student implements Cloneable {  
    public static void main(String[] args) {  
        int num = 2;  
        switch (num){  
            case 0:  
                System.out.println("number is 0");  
                break;  
            case 1:  
                System.out.println("number is 1");  
                break;  
            default:  
                System.out.println(num);  
        }  
    }  
}
```

Output:

2

While using switch statements, we must notice that the case expression will be of the same type as the variable. However, it will also be a constant value. The switch permits only int, string, and Enum type variables to be used.

Loop Statements

In programming, sometimes we need to execute the block of code repeatedly while some condition evaluates to true. However, loop statements are used to execute the set of

instructions in a repeated order. The execution of the set of instructions depends upon a particular condition.

In Java, we have three types of loops that execute similarly. However, there are differences in their syntax and condition checking time.

1. for loop
2. while loop
3. do-while loop

Let's understand the loop statements one by one.

Java for loop

In Java, for loop is similar to C and C++. It enables us to initialize the loop variable, check the condition, and increment/decrement in a single line of code. We use the for loop only when we exactly know the number of times, we want to execute the block of code.

```
for(initialization, condition, increment/decrement) {  
    //block of statements  
}
```

Consider the following example to understand the proper functioning of the for loop in java.

```
public class Calculattion {  
    public static void main(String[] args) {  
        int sum = 0;  
        for(int j = 1; j<=10; j++) {  
            sum = sum + j;  
        }  
        System.out.println("The sum of first 10 natural numbers is " + sum);  
    }  
}
```

Output:

```
The sum of first 10 natural numbers is 55
```

Java for-each loop

Java provides an enhanced for loop to traverse the data structures like array or collection. In the for-each loop, we don't need to update the loop variable. The syntax to use the for-each loop in java is given below.

```
for(data_type var : array_name/collection_name){  
    //statements  
}
```

Consider the following example to understand the functioning of the for-each loop in Java.

```
public class Calculation {  
    public static void main(String[] args) {  
        String[] names = {"Java","C","C++","Python","JavaScript"};  
        System.out.println("Printing the content of the array names:\n");  
        for(String name:names) {  
            System.out.println(name);  
        }  
    } }  
}
```

Output:

```
Printing the content of the array names:  
Java  
C  
C++  
Python  
JavaScript
```

Java while loop

The while loop is also used to iterate over the number of statements multiple times. However, if we don't know the number of iterations in advance, it is recommended to use a while loop. Unlike for loop, the initialization and increment/decrement doesn't take place inside the loop statement in while loop.

It is also known as the entry-controlled loop since the condition is checked at the start of the loop. If the condition is true, then the loop body will be executed; otherwise, the statements after the loop will be executed.

The syntax of the while loop is given below.

```
while(condition){  
    //looping statements  
}
```

Consider the following example.

```
public class Calculation {  
    public static void main(String[] args) {
```

```
int i = 0;
System.out.println("Printing the list of first 10 even numbers \n");
while(i<=10) {
System.out.println(i);
i = i + 2;
} }
}
```

Output:

```
Printing the list of first 10 even numbers
0
2
4
6
8
10
```

Java do-while loop

The do-while loop checks the condition at the end of the loop after executing the loop statements. When the number of iteration is not known and we have to execute the loop at least once, we can use do-while loop.

It is also known as the exit-controlled loop since the condition is not checked in advance. The syntax of the do-while loop is given below.

```
do
{
//statements
} while (condition);
```

Consider the following example to understand the functioning of the do-while loop in Java.

```
public class Calculation {
public static void main(String[] args) {
int i = 0;
System.out.println("Printing the list of first 10 even numbers \n");
do {
System.out.println(i);
i = i + 2;
}while(i<=10);
}
}
```

Output:

```
Printing the list of first 10 even numbers
0
2
4
6
8
10
```

Jump Statements

Jump statements are used to transfer the control of the program to the specific statements. In other words, jump statements transfer the execution control to the other part of the program. There are two types of jump statements in Java, i.e., break and continue.

Java break statement

As the name suggests, the break statement is used to break the current flow of the program and transfer the control to the next statement outside a loop or switch statement. However, it breaks only the inner loop in the case of the nested loop.

The break statement cannot be used independently in the Java program, i.e., it can only be written inside the loop or switch statement.

The break statement example with for loop

```
public class BreakExample {
    public static void main(String[] args) {
        for(int i = 0; i<= 10; i++) {
            System.out.println(i);
            if(i==6) {
                break;
            }
        }
    }
}
```

Output:

```
0
1
2
3
4
5
6
```

Java continue statement

Unlike break statement, the continue statement doesn't break the loop, whereas, it skips the specific part of the loop and jumps to the next iteration of the loop immediately.

Consider the following example to understand the functioning of the continue statement in Java.

```
public class ContinueExample {  
    public static void main(String[] args) {  
  
        for(int i = 0; i<= 2; i++) {  
  
            for (int j = i; j<=5; j++) {  
  
                if(j == 4) {  
                    continue;  
                }  
                System.out.println(j);  
            }  
        }  
    }  
}
```

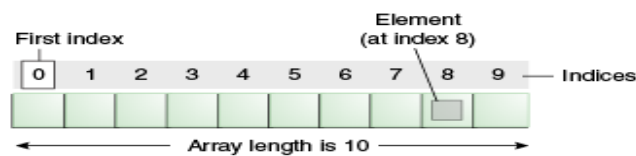
Output:

```
0  
1  
2  
3  
5  
1  
2  
3  
5  
2  
3  
5
```

Arrays

Java array is an object which contains elements of a similar data type. Additionally, The elements of an array are stored in a contiguous memory location. It is a data structure where we store similar elements. We can store only a fixed set of elements in a Java array.

Array in Java is index-based, the first element of the array is stored at the 0th index, 2nd element is stored on 1st index and so on.



Advantages

- **Code Optimization:** It makes the code optimized, we can retrieve or sort the data efficiently.
- **Random access:** We can get any data located at an index position.

Types of Array in java

There are two types of array.

- Single Dimensional Array
- Multidimensional Array

Single Dimensional Array

Syntax to Declare an Array in Java

```
dataType[] arr; (or)
```

```
dataType arr[];
```

Declaration, Instantiation and Initialization of Java Array

```
class Testarray1
{
    public static void main(String args[])
    {
        int a[]={33,3,4,5}; //declaration, instantiation and initialization
        //printing array
    }
}
```

```
for(int i=0;i<a.length;i++)  
System.out.println(a[i]);  
}}
```

Output:

```
33  
3  
4  
5
```

Multidimensional Array

In such case, data is stored in row and column based index (also known as matrix form).

Syntax to Declare Multidimensional Array in Java

```
dataType[][] arrayRefVar; (or)  
dataType arrayRefVar[][]; (or)  
dataType []arrayRefVar[];
```

Example to instantiate Multidimensional Array in Java

```
int[][] arr=new int[3][3]; //3 row and 3 column
```

Example to initialize Multidimensional Array in Java

```
arr[0][0]=1;  
arr[0][1]=2;  
arr[0][2]=3;  
arr[1][0]=4;  
arr[1][1]=5;  
arr[1][2]=6;  
arr[2][0]=7;  
arr[2][1]=8;  
arr[2][2]=9;
```

Example of Multidimensional Java Array

Let's see the simple example to declare, instantiate, initialize and print the 2Dimensional array.

MODULE-1

```
//Java Program to illustrate the use of multidimensional array
class Testarray3{
public static void main(String args[]){
//declaring and initializing 2D array
int arr[][]={{1,2,3},{2,4,5},{4,4,5}};
//printing 2D array
for(int i=0;i<3;i++)
{
for(int j=0;j<3;j++)
{
System.out.print(arr[i][j]+" ");
}
System.out.println();
}
}}
```

Output:

```
1 2 3
2 4 5
4 4 5
```